

3.1_pyomo1

March 3, 2025

1 Example 1

```
[1]: import pyomo.environ as pyo # import pyomo package
```

Every Pyomo model starts with the above. It tells Python to load the Pyomo Modeling Environment.

```
[2]: # Create an instance of a Concrete model
model = pyo.ConcreteModel()
```

The above defines a variable “model” to hold the model we are about to construct.

Concrete models are immediately constructed and data must be present at the time components are defined.

```
[3]: # Build the decision variables
model.x = pyo.Var({1,2})
```

```
[4]: # Build the objective function
model.obj = pyo.Objective(expr = 2*model.x[1]+3*model.x[2], sense = pyo.
    ↪maximize)
```

```
[5]: # Describe the constraints
model.con1 = pyo.Constraint(expr = model.x[1]-2*model.x[2] <= 4)
model.con2 = pyo.Constraint(expr = 2*model.x[1]+model.x[2] <= 18)
model.con3 = pyo.Constraint(expr = model.x[2] <= 10)
model.con4 = pyo.Constraint(expr = model.x[1] >= 0)
model.con5 = pyo.Constraint(expr = model.x[2] >= 0)
```

```
[6]: # Print the model
print()
model.pprint()
```

1 Var Declarations

```
  x : Size=2, Index={1, 2}
     Key : Lower : Value : Upper : Fixed : Stale : Domain
       1 :   None :   None :   None : False :   True :  Reals
       2 :   None :   None :   None : False :   True :  Reals
```

1 Objective Declarations

```
obj : Size=1, Index=None, Active=True
      Key : Active : Sense      : Expression
      None :      True : maximize : 2*x[1] + 3*x[2]
```

5 Constraint Declarations

```
con1 : Size=1, Index=None, Active=True
      Key : Lower : Body      : Upper : Active
      None : -Inf : x[1] - 2*x[2] : 4.0 :      True
con2 : Size=1, Index=None, Active=True
      Key : Lower : Body      : Upper : Active
      None : -Inf : 2*x[1] + x[2] : 18.0 :      True
con3 : Size=1, Index=None, Active=True
      Key : Lower : Body : Upper : Active
      None : -Inf : x[2] : 10.0 :      True
con4 : Size=1, Index=None, Active=True
      Key : Lower : Body : Upper : Active
      None : 0.0 : x[1] : +Inf :      True
con5 : Size=1, Index=None, Active=True
      Key : Lower : Body : Upper : Active
      None : 0.0 : x[2] : +Inf :      True
```

7 Declarations: x obj con1 con2 con3 con4 con5

```
[7]: # Solve the model through the GLPK solver
      solver = pyo.SolverFactory('glpk')
      solver.solve(model)
```

```
[7]: {'Problem': [{'Name': 'unknown', 'Lower bound': 38.0, 'Upper bound': 38.0,
'Number of objectives': 1, 'Number of constraints': 5, 'Number of variables': 2,
'Number of nonzeros': 7, 'Sense': 'maximize'}], 'Solver': [{'Status': 'ok',
'Termination condition': 'optimal', 'Statistics': {'Branch and bound': {'Number
of bounded subproblems': 0, 'Number of created subproblems': 0}}, 'Error rc': 0,
'Time': 0.00946807861328125}], 'Solution': [OrderedDict({'number of solutions':
0, 'number of solutions displayed': 0})]}
```

```
[8]: # Print the optimization results
      print()
      model.display() # List of all optimization results
      print()
      print('Optimal value: ', pyo.value(model.obj)) # Print the value of model.obj
      ↪(i.e., optimal objective value)
```

Model unknown

Variables:

```
x : Size=2, Index={1, 2}
```

	Key	: Lower	: Value	: Upper	: Fixed	: Stale	: Domain
1	: None	: 4.0	: None	: False	: False	: Reals	
2	: None	: 10.0	: None	: False	: False	: Reals	

Objectives:

```
obj : Size=1, Index=None, Active=True
    Key : Active : Value
    None : True : 38.0
```

Constraints:

```
con1 : Size=1
      Key : Lower : Body : Upper
      None : None : -16.0 : 4.0
con2 : Size=1
      Key : Lower : Body : Upper
      None : None : 18.0 : 18.0
con3 : Size=1
      Key : Lower : Body : Upper
      None : None : 10.0 : 10.0
con4 : Size=1
      Key : Lower : Body : Upper
      None : 0.0 : 4.0 : None
con5 : Size=1
      Key : Lower : Body : Upper
      None : 0.0 : 10.0 : None
```

Optimal value: 38.0

1.1 Alternative options for the toy example

```
[9]: import pyomo.environ as pyo
model_alt = pyo.ConcreteModel()

[10]: # We can build multiple decision variables
      # We can build decision variables with speical domains and/or explicit bounds
model_alt.x1 = pyo.Var(within=pyo.NonNegativeReals)
model_alt.x2 = pyo.Var(bounds=(0,10))

[11]: # If 'sense' is omitted, the default is minimization.
model_alt.obj = pyo.Objective(expr = -2*model_alt.x1 - 3*model_alt.x2)

[12]: # We can build a constraint list
      # 'expr' can be a 3-tuple (LB, expr, UB)
model_alt.cons = pyo.ConstraintList()
model_alt.cons.add(model_alt.x1-2*model_alt.x2 <= 4)
model_alt.cons.add((None, 2*model_alt.x1+model_alt.x2, 18))
```

```
[12]: <pyomo.core.base.constraint.ConstraintData at 0x2127cd65c70>
```

```
[13]: solver = pyo.SolverFactory('glpk')
solver.solve(model_alt)
print()
model_alt.display()
print()
print('Optimal value: ', -pyo.value(model_alt.obj))
```

Model unknown

Variables:

```
x1 : Size=1, Index=None
     Key : Lower : Value : Upper : Fixed : Stale : Domain
     None :      0 :   4.0 :   None : False : False : NonNegativeReals
x2 : Size=1, Index=None
     Key : Lower : Value : Upper : Fixed : Stale : Domain
     None :      0 :  10.0 :    10 : False : False : Reals
```

Objectives:

```
obj : Size=1, Index=None, Active=True
     Key : Active : Value
     None :   True : -38.0
```

Constraints:

```
cons : Size=2
      Key : Lower : Body : Upper
      1 :  None : -16.0 :   4.0
      2 :  None :  18.0 :  18.0
```

Optimal value: 38.0

```
[ ]:
```

brewery

March 3, 2025

1 Example 2: The brewery problem

The following models are partially flexible: we assume that the resources are always malt, hops, and yeast.

1.1 A concrete model

```
[1]: import pyomo.environ as pyo

[2]: # Define the index set
Beer = {'Light', 'Dark', 'Ale', 'Premium'}

[3]: # Data
w_malt = {'Light':1, 'Dark':1, 'Ale':0, 'Premium':3}
w_hops = {'Light':2, 'Dark':1, 'Ale':2, 'Premium':2}
w_yeast = {'Light':1, 'Dark':1, 'Ale':1, 'Premium':4}
revenue = {'Light':6, 'Dark':5, 'Ale':3, 'Premium':7}
avail_malt = 50
avail_hops = 150
avail_yeast = 80

[4]: # Create a Concrete model
model = pyo.ConcreteModel()

[5]: # Build the decision variables
model.x = pyo.Var(Beer, within=pyo.NonNegativeReals)

[6]: # Build the objective function
def obj_value_rule(model):
    return sum(revenue[b] * model.x[b] for b in Beer)

model.obj = pyo.Objective(rule=obj_value_rule, sense = pyo.maximize)

[7]: # Build the constraints
model.malt = pyo.Constraint(expr = sum(w_malt[i]*model.x[i] for i in Beer) <=
    ↪ avail_malt)
model.hops = pyo.Constraint(expr = sum(w_hops[i]*model.x[i] for i in Beer) <=
    ↪ avail_hops)
```

```
model.yeast = pyo.Constraint(expr = sum(w_yeast[i]*model.x[i] for i in Beer) <=
↪avail_yeast)
```

```
[8]: # Solve the model through the GLPK solver
```

```
from pyomo.opt import SolverFactory
```

```
solver = SolverFactory('glpk')
```

```
results = solver.solve(model)
```

```
[9]: # Print the optimization results
```

```
print()
```

```
model.display() # List of all optimization results
```

```
print()
```

```
print('Optimal value: ', pyo.value(model.obj)) # Print the value of model.obj
```

```
↪(i.e., optimal objective value)
```

Model unknown

Variables:

```
x : Size=4, Index={Dark, Premium, Light, Ale}
```

```
Key      : Lower : Value : Upper : Fixed : Stale : Domain
```

```
Ale      :      0 :  30.0 :   None : False : False : NonNegativeReals
```

```
Dark     :      0 :  10.0 :   None : False : False : NonNegativeReals
```

```
Light    :      0 :  40.0 :   None : False : False : NonNegativeReals
```

```
Premium  :      0 :   0.0 :   None : False : False : NonNegativeReals
```

Objectives:

```
obj : Size=1, Index=None, Active=True
```

```
Key   : Active : Value
```

```
None  :   True : 380.0
```

Constraints:

```
malt : Size=1
```

```
Key   : Lower : Body : Upper
```

```
None  :   None : 50.0 :  50.0
```

```
hops : Size=1
```

```
Key   : Lower : Body : Upper
```

```
None  :   None : 150.0 : 150.0
```

```
yeast : Size=1
```

```
Key   : Lower : Body : Upper
```

```
None  :   None :  80.0 :  80.0
```

Optimal value: 380.0

```
[10]: # Print the optimal solution
```

```
for i in Beer:
```

```
    print(i, pyo.value(model.x[i]))
```

Dark 10.0
Premium 0.0
Light 40.0
Ale 30.0

1.2 An abstract model

```
[11]: import pyomo.environ as pyo
```

```
[12]: # Create an Abstract model
model = pyo.AbstractModel()
```

```
[13]: # Build the index set
model.Beer = pyo.Set()
```

```
[14]: # Build parameters
model.w_malt = pyo.Param(model.Beer, within=pyo.NonNegativeReals)
model.w_hops = pyo.Param(model.Beer, within=pyo.NonNegativeReals)
model.w_yeast = pyo.Param(model.Beer, within=pyo.NonNegativeReals)
model.revenue = pyo.Param(model.Beer, within=pyo.NonNegativeReals)
model.avail_malt = pyo.Param(within=pyo.NonNegativeReals)
model.avail_hops = pyo.Param(within=pyo.NonNegativeReals)
model.avail_yeast = pyo.Param(within=pyo.NonNegativeReals)
```

```
[15]: # Build the decision variables
model.x = pyo.Var(model.Beer, within=pyo.NonNegativeReals)
```

```
[16]: # Build the objective function
def obj_value_rule(model):
    return sum(model.revenue[i]*model.x[i] for i in model.Beer)

model.obj = pyo.Objective(rule=obj_value_rule, sense = pyo.maximize)
```

```
[17]: # Build the malt constraint
def malt_rule(model):
    return sum(model.w_malt[i]*model.x[i] for i in model.Beer) <= model.
    ↪ avail_malt

model.malt = pyo.Constraint(rule=malt_rule)
```

```
[18]: # Build the hops constraint
def hops_rule(model):
    return sum(model.w_hops[i]*model.x[i] for i in model.Beer) <= model.
    ↪ avail_hops

model.hops = pyo.Constraint(rule=hops_rule)
```

```
[19]: # Build the yeast constraint
def yeast_rule(model):
    return sum(model.w_yeast[i] * model.x[i] for i in model.Beer) <= model.
    ↪avail_yeast

model.yeast = pyo.Constraint(rule=yeast_rule)

[20]: # Load data
instance = model.create_instance('brewery.dat')

[21]: # Solve the model through the GLPK solver
solver = pyo.SolverFactory('glpk')
solver.solve(instance)

[21]: {'Problem': [{'Name': 'unknown', 'Lower bound': 380.0, 'Upper bound': 380.0,
'Number of objectives': 1, 'Number of constraints': 3, 'Number of variables': 4,
'Number of nonzeros': 11, 'Sense': 'maximize'}], 'Solver': [{'Status': 'ok',
'Termination condition': 'optimal', 'Statistics': {'Branch and bound': {'Number
of bounded subproblems': 0, 'Number of created subproblems': 0}}, 'Error rc': 0,
'Time': 0.011029243469238281}], 'Solution': [OrderedDict({'number of solutions':
0, 'number of solutions displayed': 0})]}
```

```
[22]: # Print the results
instance.display()
print()
print('Optimal objective value: ', pyo.value(instance.obj))
```

Model unknown

Variables:

```
x : Size=4, Index=Beer
  Key      : Lower : Value : Upper : Fixed : Stale : Domain
  Ale      :      0 :  30.0 :   None : False : False : NonNegativeReals
  Dark     :      0 :  10.0 :   None : False : False : NonNegativeReals
  Light    :      0 :  40.0 :   None : False : False : NonNegativeReals
  Premium  :      0 :   0.0 :   None : False : False : NonNegativeReals
```

Objectives:

```
obj : Size=1, Index=None, Active=True
  Key : Active : Value
  None : True : 380.0
```

Constraints:

```
malt : Size=1
  Key : Lower : Body : Upper
  None : None : 50.0 :  50.0
hops : Size=1
  Key : Lower : Body : Upper
```



```
None : None : 150.0 : 150.0
yeast : Size=1
Key : Lower : Body : Upper
None : None : 80.0 : 80.0
```

Optimal objective value: 380.0

```
[23]: # Print the optimal solution
      for i in instance.Beer:
          print(i, pyo.value(instance.x[i]))
```

```
Light 40.0
Dark 10.0
Ale 30.0
Premium 0.0
```